

GB3 Group 5 - Documentation and Results

PPA Automation Scripts

Automated configuration sweep, build, testing and analysis for the PicoRV32 iCEBreaker design

The project includes a small automation flow for exploring the power, performance and area trade-offs of different PicoRV32 and cache configurations. The scripts generate a set of design configurations, build each configuration in an isolated directory, run functional tests, collect synthesis and timing metrics, and generate plots showing the resulting trade-offs.

The flow is split into four scripts:

- `interactive_ppa.py` : interactively generates `configs.csv`.
- `run_ppa_fixed.py` : builds and measures each configuration.
- `analyse_ppa.py` : post-processes the collected results into tables and plots.
- `run_all.sh` : runs the full project-level simulation and benchmarking flow.

Together, these scripts make it possible to compare CPU parameters, cache sizes and cache implementations systematically, rather than changing Verilog parameters manually and rebuilding one design at a time.

How the flow works

The user normally starts with:

```
cd picosoc
python3 interactive_ppa.py
```

The interactive script asks which parameters should be fixed, varied, or skipped. For binary CPU parameters, the accepted choices are:

- `0` : force the parameter to zero.
- `1` : force the parameter to one.
- `v` : vary the parameter across both zero and one.
- `s` : skip the parameter and leave the Verilog default unchanged.

The script supports the main PicoRV32 feature parameters, including the barrel shifter, multiplier, divider, counters, IRQ support, two-cycle ALU and comparison options, register-file configuration, misalignment and illegal-instruction catching, latched memory data, and the optional clock divider.

It also supports non-binary design choices:

- `MEM_WORDS` , either as a single value or a `start:end:step` sweep.
- `CACHE_LINES` , selected from 16, 32, 64 or 128.
- `CACHE_WORDS_PER_LINE` , selected from 1, 2, 4, 8 or 16.
- Cache module selection, parsed directly from `icache.v` , with an additional option for no instruction cache.

The selected options are expanded into a Cartesian product of configurations. Each row is written to `configs.csv`, with a generated `config_name` that records the active parameter choices. This naming convention makes the output directories and final results easier to trace back to the configuration that produced them.

Per-configuration build and testing: `run_ppa_fixed.py`

`run_ppa_fixed.py` is the main measurement script. It reads `configs.csv` and processes one configuration at a time. For each configuration, it creates a clean build directory under:

```
picosoc/build_fixed/<config_name>/
```

The script then copies the Verilog sources and test infrastructure into that directory. This includes the SoC files, cache files, data-memory lookahead buffer, `picorv32.v`, the Makefile, firmware, RISC-V tests, scripts, divider testbench and I-cache testbench. Keeping each configuration in a separate build directory avoids cross-contamination between runs and preserves the intermediate logs for debugging.

Parameter replacement is then applied to the copied files. The script modifies:

- `icebreaker.v`, for top-level feature parameters and the optional clock divider.
- `picosoc.v`, for SoC-level settings such as counters, IRQs, cache enable, data-memory lookahead, cache geometry and cache module selection.
- `picorv32.v`, for internal CPU parameters such as two-stage shift, two-cycle ALU, two-cycle compare, register-file options and exception-catching options.
- `testbench.v`, so that the simulation testbench matches the hardware configuration being built.

If multiplication or division is enabled, the script also enables PCPI in the testbench so that the relevant RV32M instruction tests are exercised correctly.

Before synthesis, the script can run three levels of functional checking:

1. The standard RV32IM instruction tests using `make test`.
2. The divider unit test using `make div_test`.
3. The I-cache unit test using `make icache_test`.

This early test stage is important because it prevents a broken configuration from being treated as a valid PPA data point. By default, if the RV32IM instruction tests fail, synthesis and place-and-route are skipped for that configuration. The output CSV still records the failed configuration, with the test-pass fields set accordingly and the PPA fields set to zero. If required, this behaviour can be overridden with:

```
python3 picosoc/run_ppa_fixed.py --continue-on-test-fail
```

For quick area/timing sweeps where functional testing has already been done, tests can also be skipped using:

```
python3 picosoc/run_ppa_fixed.py --skip-tests
```

PPA measurement

For configurations that proceed past the test stage, the script runs the iCE40 build flow:

1. Yosys synthesis with `synth_ice40` and `stat -width`.
2. A second Yosys pass to produce `design.json`.
3. `nextpnr-ice40` placement and routing for the UP5K SG48 target.
4. `icetime` timing analysis.
5. Parsing of synthesis, placement and timing outputs.

The extracted metrics include:

- LUT count.
- Flip-flop count.
- BRAM count.
- DSP count.
- Carry-cell count.
- Estimated maximum clock frequency.
- Number of logic levels.
- Logic-cell, RAM, DSP, I/O, PLL and global-buffer utilisation.

The final results are written to:

```
picosoc/ppa_config_fixed.csv
```

Each row contains the configuration name, test pass/fail status, PPA metrics and the parameters used for that run.

Analysis script: `analyse_ppa.py`

Once `ppa_config_fixed.csv` has been generated, `analyse_ppa.py` summarises the design-space exploration results. It reads the CSV file and writes analysis outputs into:

```
picosoc/ppa_analysis_output/
```

The script produces:

- `correlation_params_metrics.csv`: correlation values between binary design parameters and PPA metrics.
- `correlation_heatmap.png`: a heatmap showing how each parameter correlates with LUTs, flip-flops, BRAMs, carry cells, Fmax and logic levels.
- `mean_differences.csv`: the mean LUT and Fmax difference between each parameter being disabled and enabled.
- `linear_regression_coefficients.csv`: fitted coefficients estimating the effect of each binary feature on LUT count and Fmax.

- `luts_per_config.png` : a sorted bar chart of LUT usage by configuration.
- `luts_vs_fmax.png` : a scatter plot showing the area/performance trade-off, coloured by the number of enabled features.

These plots make it easier to see which features cost the most area, which parameters reduce timing performance, and which configurations offer the best trade-off between resource use and Fmax.

Top-level script: `run_all.sh`

`run_all.sh` is a convenience script for running the broader project workflow from the top-level repository directory. It first sources the OSS CAD Suite environment if it is available, then sets the RISC-V cross-compiler prefix.

The script runs:

1. The interactive PPA flow from the `picosoc/` directory.
2. The standard instruction tests using the top-level `make test`.
3. The Dhrystone benchmark from the `dhrystone/` directory.
4. An optional custom benchmark based on `picosoc/integral.c`, if that file exists.

For the custom benchmark, the script compiles `integral.c` into an ELF file, converts it into a Verilog hex file, temporarily replaces `icebreaker_fw.hex`, runs the SoC simulation, and then restores the original firmware hex file.

The script also includes a commented-out hardware programming section. This can be enabled when the iCEBreaker board is connected, but the current version of the script does not automatically run the hardware benchmark or collect UART CPI results.

Expected environment

The automation flow expects:

- OSS CAD Suite available, including Yosys, nextpnr, icetime, icepack and related iCE40 tools.
- A RISC-V embedded GCC toolchain in the path.
- Icarus Verilog and the existing PicoRV32 Makefile test infrastructure.
- The project directory structure used by the GB3 submission, with `picosoc/` containing the SoC files and `picorv32.v` located one level above it.

Important notes and limitations

The parameter overrides are implemented using regular-expression replacement on copied Verilog files. This is simple and effective for the known codebase, but it assumes that the relevant parameter declarations and module instantiations keep the expected formatting. If the Verilog is heavily refactored, the replacement expressions should be checked.

`run_ppa_fixed.py` deletes and recreates the `build_fixed/` directory at the start of each run. Any logs or intermediate build products that need to be kept should be copied elsewhere before rerunning the script.

`analyse_ppa.py` currently assumes a fixed list of binary parameters. If new parameters are added to `interactive_ppa.py`, the analysis script should be updated so that the new columns are included in the correlation, mean-difference and regression analysis.

The current top-level `run_all.sh` script runs simulations and an optional custom benchmark, but the hardware benchmark flow is not active by default. The hardware programming section is present only as commented-out code.